

Module: **Problem 9**

This module provides a utility to check whether two collections contain the same elements (i.e., are permutations of each other).
If they are permutations, the function also reports the element that appears most frequently, how many times it occurs, and its Python type.

Functions

`is_list_permutation(L1, L2)`

Description

Determines whether `L1` and `L2` are permutations of one another.

A permutation means the two inputs contain exactly the same elements with the same multiplicities, regardless of order.

If they are not permutations, the function returns `False`.

* If they are permutations, it returns a three-item tuple:

1. the element that occurs most often,
2. the number of occurrences of that element, and
3. the element's type (`type(value)`).

When both inputs are empty, the tuple `(None, None, None)` is returned.

Parameters

- `L1` (*iterable of hashable objects*): The first collection to compare.
- `L2` (*iterable of hashable objects*): The second collection to compare.

Returns

- `False` if `L1` and `L2` are **not** permutations of each other.
- `tuple(value, count, type)` where `value` is the most frequent element, `count` is its frequency, and `type` is `type(value)`, when the inputs are permutations.
- `(None, None, None)` when both inputs are empty.

Examples

```
# Example 1 - strings are treated as iterables of characters
result = is_list_permutation('test', 'test')
# → ('t', 2, <class 'str'>) # 't' appears twice, most frequent character

# Example 2 - empty strings (empty iterables)
result = is_list_permutation('', '')
# → (None, None, None)

# Example 3 - proper list inputs that are permutations
result = is_list_permutation([1, 2, 2, 3], [2, 3, 2, 1])
# → (2, 2, <class 'int'>) # 2 occurs twice, more than any other element

# Example 4 - non-permutations
```

```
result = is_list_permutation([1, 2, 3], [1, 2, 4])  
# → False
```

Edge Cases / Notes - Non-iterable arguments (e.g., passing an integer such as `0` or `1`) raise a `TypeError` because `Counter` expects an iterable:

```
is_list_permutation(0, 0) # TypeError: 'int' object is not iterable
```

- Passing **strings** works (strings are iterables of characters), but the function will count characters rather than whole strings. This can lead to surprising results if the caller expects whole-string comparison. - The function relies on `collections.Counter`; therefore, all elements must be **hashable**. - When both inputs are empty, the function intentionally returns `(None, None)` instead of `False`.